

Prompts for the STUDENT EXCUSE GENERATOR:



Write a professional C++ console application called "Student Excuse Generator".

Requirements:

1. Take student name as input.
2. Store at least 10 creative excuse templates in a vector.
3. Use randomization to generate different excuses each time.
4. Replace the name with the actual student name.
5. Use class-based programming with proper functions.
6. Add beginner-friendly comments.
7. Handle invalid or empty input properly.
8. Display output in a clean console UI format.
9. Allow the user to generate multiple excuses without restarting the program.
10. Follow professional coding standards and proper indentation.



Generate more creative and funny excuse templates for a C++ Student Excuse Generator project.

Conditions:

1. Use the placeholder {name}.
2. Excuses should sound realistic but humorous.
3. Make excuses student and technology related.
4. Avoid offensive content.
5. Generate at least 15 unique excuse templates.



Refactor the C++ Student Excuse Generator program into a more professional structure.

Requirements:

1. Create a separate class named "Student Excuse Generator".
2. Use private and public access specifiers correctly.
3. Create a dedicated function for placeholder replacement.
4. Improve variable and function naming conventions.
5. Keep the code beginner-friendly and modular.
6. Improve overall readability and maintainability.



The Student Excuse Generator program is not handling empty input correctly.

Fix the issue by:

1. Detecting empty input from the user.
2. Displaying an error message for invalid input.
3. Preventing the program from generating excuses with blank names.
4. Maintaining professional coding standards.



Improve the random roast generation logic in the C++ program so the output feels more dynamic and less repetitive.

Requirements:

1. Use proper random seeding.
2. Ensure different roasts appear on different runs.
3. Keep the code beginner-friendly.
4. Maintain performance and readability.



Refactor the entire C++ AI Roast Generator project into a final professional version suitable for college assignment submission.

Requirements:

1. Professional formatting and indentation.
2. Class-based design.
3. Functions with clear responsibilities.
4. Proper input validation.
5. Clean console UI.
6. Multiple roast generation support.
7. Maintain beginner readability.
8. Add meaningful roast templates.
9. Ensure the program compiles without warnings or errors.